

Université Badji Mokhtar Annaba
Faculté de l'ingéniorat
Département d'informatique
Master 1, Réseaux et sécurité Informatique

Corba

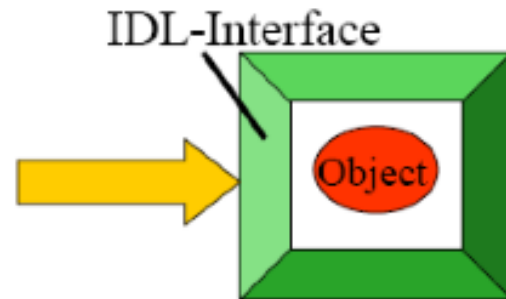
Common Object Request Broker Architecture

Préparé par: MAA. Yahiouche Salima

Corba: Definition, caractéristiques

- Un standard (pas un produit), qui spécifie comment les objets communiquent entre eux dans un réseau.
- Indépendant des plateformes
- Indépendant des langages
- Une architecture ouverte, appuyé sur un langage neutre: IDL Interface Definition Language
- Chaque langage de programmation supportant Corba implémente une IDL
- Java IDL ou idlj est l'implémentation IDL pour Java.
- Corba et IDL sont issus d'un consortium de trusts mondiaux (notamment IBM, Boeing, Sun, etc.) connus sous le nom de OMG (Object Management Group),

Corba travaille avec les interfaces



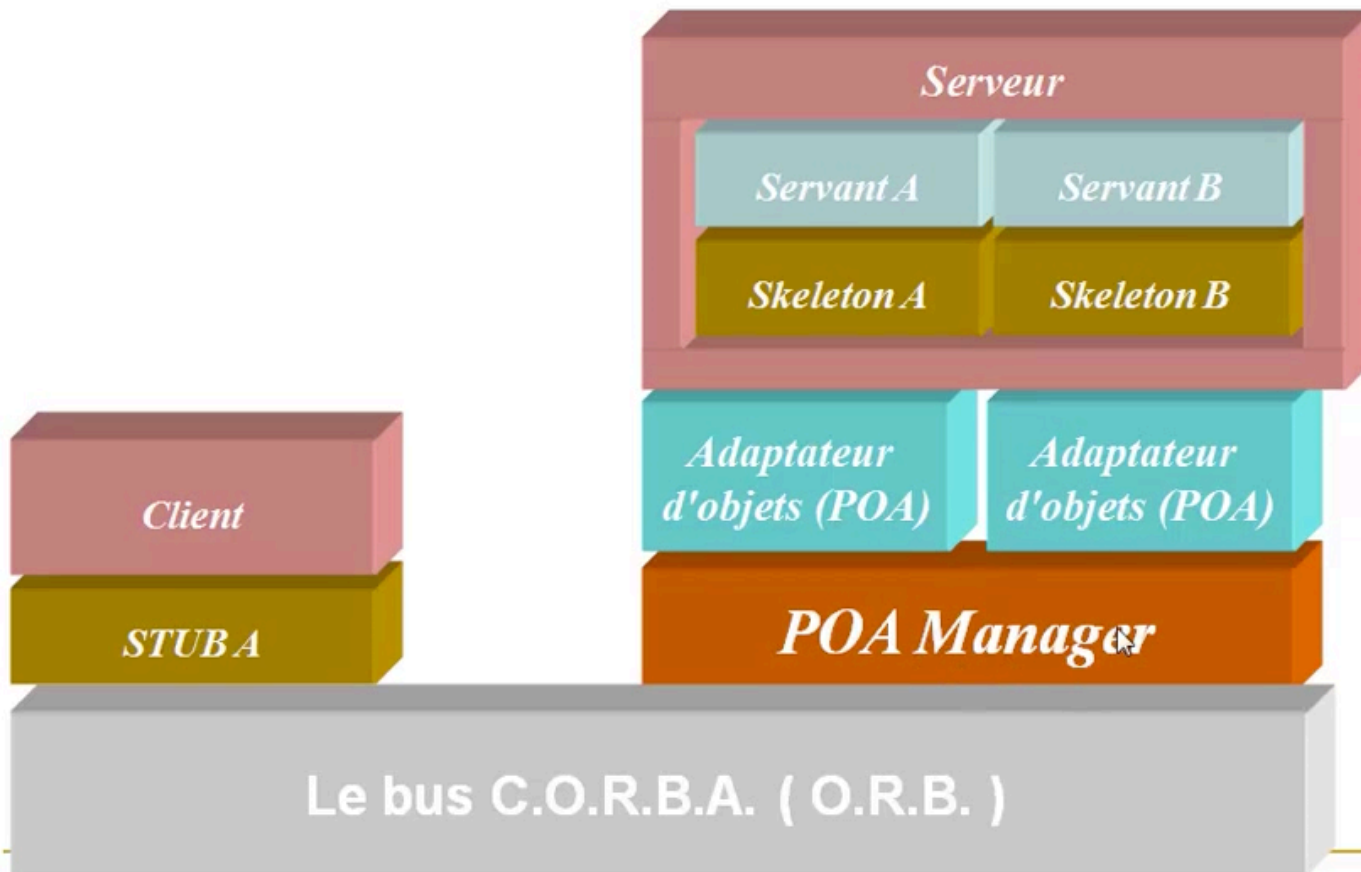
- Les objets Corba sont accessibles seulement via les interfaces
- La séparation de l'interface et de l'implémentation permet plusieurs implémentations pour une même interface

The diagram illustrates the CORBA architecture, showing the interaction between a client application and a servant object through the Object Request Broker (ORB). The client application interacts with the Dynamic Invocation Interface (DII) or IDL stub. The servant object interacts with the ORB interface, IDL skeleton, Dynamic Skeleton Interface (DSI), or several possible interfaces. The ORB interface and IDL skeleton are connected to the ORB. The DSI and several possible interfaces are connected to the ORB. The ORB is connected to the Interface repository and the Implementation repository.

Legend:

- standard interface (red hatched box)
- several possible interfaces (black hatched box)
- proprietary ORB interface (blue box)
- application-dependent interface (grey hatched box)

architecture CORBA simplifiée



L'architecture CORBA: ORB

- Pour permettre des interactions entre les objets de différents programmes d'un réseau, Java IDL "fournit" un ORB (Object Request Broker).
- Un ORB est une bibliothèque de classes qui permet une communication de bas niveau entre Java IDL et d'autres applications basées sur Corba.

Orb

- Un **object request broker** : « courtier de requêtes objet », ou **ORB**, est un ensemble de fonctions (classes Java, bibliothèques C++...) qui implémentent un bus logiciel par lequel des objets envoient et reçoivent des requêtes et des réponses, de manière transparente et portable
- Un ORB s'apparente à une tuyauterie permettant les échanges de messages entre objets. Les ORB appartiennent à la famille des [middleware](#) (ou intergiciels).

L'architecture CORBA: coté client

- Du côté du client, l'application a une référence vers l'objet distribué. La référence a une méthode "stub" qui est le remplaçant pour la méthode appelée à distance.
- Le "stub" est installé dans l'ORB, l'appeler revient donc à activer les fonctions de communication de l'ORB qui se chargera de la connexion au serveur.

L'architecture CORBA: coté serveur

- Du côté du serveur, l'ORB utilise un "squelette" de code pour traduire l'appel à distance en un appel de méthode sur un objet local.
- Le squelette se charge de bien traduire la requête dans le format attendu par l'objet local. Quand la méthode finit (ie: quand elle retourne) le squelette se charge de traduire la valeur de retour ou les erreurs et de les transmettre à l'ORB qui les transmet à son tour au client.

L'architecture CORBA: protocole de communication

- Entre les ORB, la communication est établie grâce à un protocole spécial: **IIOP** (Internet Inter-ORB Protocol).
- IIOP, basé sur le standard TCP/IP, définit comment des ORB pour Corba échangent des informations.
- Tout comme Corba et IDL, IIOP est un standard définit par l'OMG (Object Management Group).

Le processus de développement d'une application Corba en java

- Deux méthodes:
- Une première classique sans services externes de nommage.
- Une deuxième plus compact, utilisant JNDI, une interface offrant Un service de nommage et un service d'annuaire

Le processus de développement d'une application Corba en java

-1- Définir l'interface à distance

- On définit l'interface à distance en utilisant le langage de définition de l'interface de l'OMG, c'est à dire IDL.
- On utilise IDL plutôt que Java car le compilateur idl-to-java travaille à partir d'un (code) source IDL, générant tous les fichiers "stub" et "squelette" avec le code d'infrastructure nécessaire à la connexion avec l'ORB.
- En utilisant IDL, on permet aux autres développeurs d'implémenter des clients et des serveurs dans d'autres langages compatibles Corba.

Le processus de développement d'une application corba en java

-2- Compiler l'interface à distance

Quand on lance le compilateur idlj sur le fichier source de définition de l'interface, il génère une version en Java de l'interface, il génère aussi les stubs et les squelettes.

Le processus de développement d'une application corba en java

-3-Implémenter le serveur

- Une fois le compilateur idlj lancé, on peut utiliser les squelettes générés pour assembler le serveur.
- En plus d'implémenter les méthodes de l'interface à distance, le code du serveur inclut un mécanisme pour lancer l'ORB et attendre une requête d'un client.

Le processus de développement d'une application corba en java

- 4 - Implémenter un client

- De même, on utilisera les stubs générés par le compilateur comme base pour l'application client.
- Le code client qui est construit à partir des stubs, lance l'ORB de son côté qui recherche une connexion vers le serveur en utilisant le service de noms indiqué par Java IDL, obtient une référence vers l'objet à distance et invoque à distance ses méthodes.

Le processus de développement d'une application corba en java

- 5 - Démarrer l'application

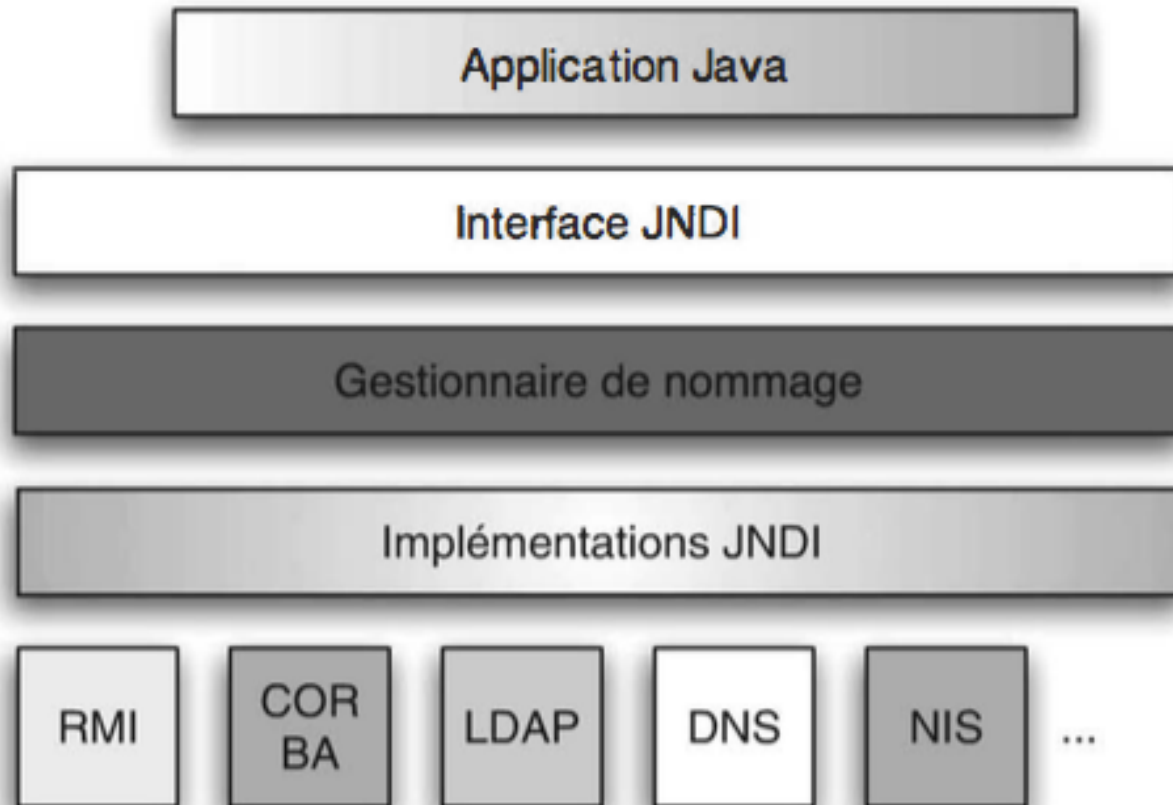
- Une fois le serveur et le client créés, on peut lancer le ***service de noms***, puis démarrer le ***serveur***, et enfin lancer le ***client***.

Implémentation rapide utilisant JNDI

(voir Annexe pour l'implémentation classique)

- ***Definition:*** JNDI(Java Naming and Directory Interface) est une interface Java qui offre au développeur deux services:
- Un service de nomage(javax.naming.*)
- Un service d'annuaire:
(javax.naming.directory.*)

Une seule API pour multiples implémentations



Etapes d'implémentations:

- les mêmes que celles vues précédemment (ce sont aussi les mêmes pour la version classique)
- Il faut **obligatoirement** créer un fichier **jndi.properties** contenant **exactement** les deux lignes suivantes:
`java.naming.factory.initial=com.sun.jndi.cosnaming.CNCtxFactory`
`java.naming.provider.url=iiop://localhost:1050`
- Les deux fichiers serveur et Client seront considérablement allégés, comparé à la version classique (voir Annexe)

1-2 .Création et Compilation du fichier IDL:

1- créer le fichier IDL

2- le compiler

Eg de fichier IDL : HelloApp.idl

```
module HelloApp
```

```
{
```

```
interface Hello
```

```
{
```

```
    string sayHello();
```

```
    oneway void shutdown();
```

```
};
```

```
}; //shutdown() pour  
arrêter l'ORB
```

La compilation se fait en
ligne de *commande(cmd*
pour windows), après
spécification du chemin:

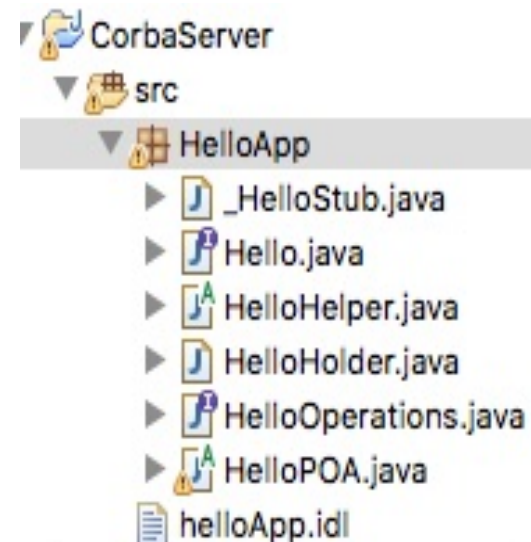
Documents/workspaceJava/CorbaSe
rver/src/> idlj -fall HelloApp.idl

Documents/workspaceJava/CorbaServer/src/> idlj -fall HelloApp.idl

**Attention ce chemin sera différent chez
vous, c'est le chemin qui mène vers où
est enregistré le fichier HelloApp.idl**

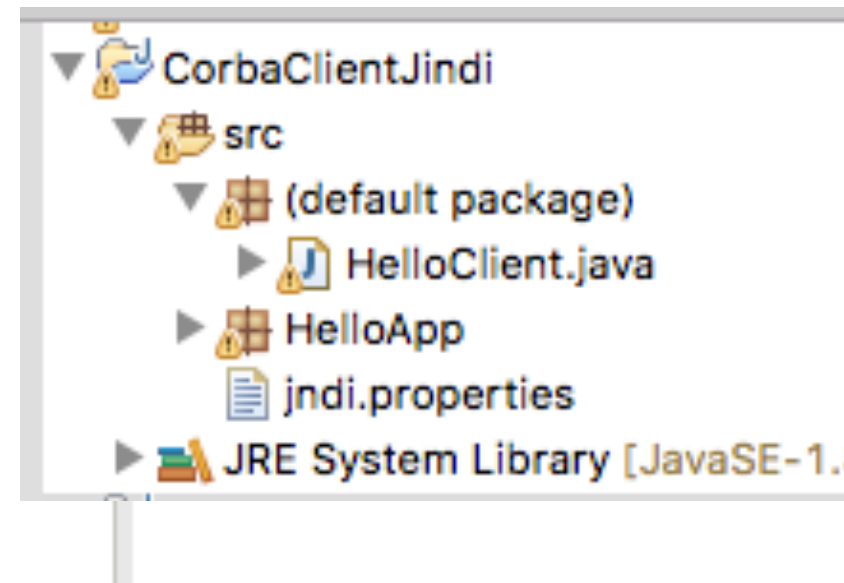
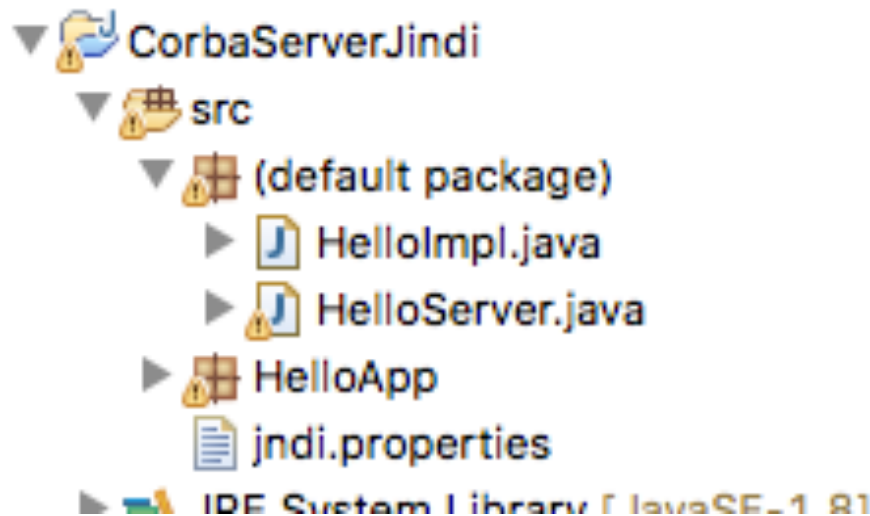
Après la compilation, ce
module est mappé en
un package:

`package HelloApp`, qui
contient les squelettes
nécessaires pour
assembler le serveur



Création et Inclusion du fichier jndi.properties:

- Le fichier jndi.properties doit être inclut aussi bien dans le projet client que dans celui du serveur



Code source Servant

```
import HelloApp.*;
import org.omg.CosNaming.*;
import org.omg.CosNaming.NamingContextPackage.*;
import org.omg.CORBA.*;
import org.omg.PortableServer.*;
import org.omg.PortableServer.POA;
import java.util.Properties;
```

```
class HelloImpl extends HelloPOA {
```

```
//Créer et initialiser une instance ORB
```

```
private ORB orb;
```

```
public void setORB(ORB orb_val) { orb = orb_val; }
```

```
// implement sayHello() method
```

```
public String sayHello() { return "\nHello world !!\n"; }
```

```
// implement shutdown() method
```

```
public void shutdown() { orb.shutdown(false); } }
```

HelloServer :

```
public class HelloServer {
```

```
    public static void main(String[] args) {
```

```
        ORB orb=ORB.init(args,null);
```

```
        try {
```

```
            Context ctx=new InitialContext();
```

```
            //il faut importer le package javaxnaming
```

```
            POA rootPoa=
```

```
                POAHelper.narrow(orb.resolve_initial_references("RootPOA"));
```

```
            rootPoa.the_POAManager().activate();
```

```
            //instancier l'objet servant:
```

```
            HelloImpl helloImpl= new HelloImpl();
```

```
            //passer la reference orb à la méthode setOrb pour que  
                ORBShutdown puisse être appelé:
```

```
            helloImpl.setOrb(orb);
```

```
        ctx.rebind("RSI2020",
```

```
            rootPoa.servant_to_reference(helloImpl));
```

```
        /* maintenant le client utilisera RSI2020 pour retrouver  
            la ref vers l'objet servant hello (voir plus loin  
            diapo client*/
```

```
        System.out.println("Hello! server waiting....");
```

```
        //attendre l'invocation du client:
```

```
        orb.run();
```

```
        /*l'ORB rentrera en attente après la fin de  
            l'invocation, c'est pour
```

```
            cela que le client doit l'arrêter lorsqu'il termine ses  
            appels, shutdown()*/
```

```
        } catch (Exception e) {
```

```
            System.err.println("Error: "+ e);
```

```
            e.printStackTrace(System.out);
```

```
        }  
    }
```

HelloClient

```
public class HelloClient {  
    static Hello helloImplstub;  
    public static void main(String[] args) {  
  
        try {  
            Context ctx=new InitialContext();  
            org.omg.CORBA.Object  
                objRef=(Object)  
                ctx.lookup("RSI2020");  
  
            // resolve the Object Reference in  
            Naming:  
            helloImplstub=HelloHelper.narrow(obj  
                Ref);
```

*//juste pour voir de quoi a l'air la
référence:*

```
System.out.println("On a obtenu un  
gestionnaire pour le serveur d'objet "  
+helloImplstub);
```

// l'appel de la méthode distante:

```
System.out.println(helloImplstu  
    b.sayHello());  
    helloImplstub.shutdown();  
} catch (Exception e) {  
    e.printStackTrace();  
}  
  
}}
```

Pour exécuter

- lancer sur une fenêtre cmd:

start orbd –ORBInitialPort 1050

Lancer le serveur

Lancer le client

(pas besoin de run configuration)

NB. je vous conseille de réviser un exemple avec une idl plus riche sur youtube du professeur Mohamed Youssfi (Tutorial Corba avec JNDI, daté 2014)

Annexe

Méthode Classique

- Sans JNDI, mais plus longue.....

1-2 .Création et Compilation du fichier IDL:

1- créer le fichier IDL

2- le compiler

Eg de fichier IDL : HelloApp.idl

```
module HelloApp
```

```
{
```

```
interface Hello
```

```
{
```

```
    string sayHello();
```

```
    oneway void shutdown();
```

```
};
```

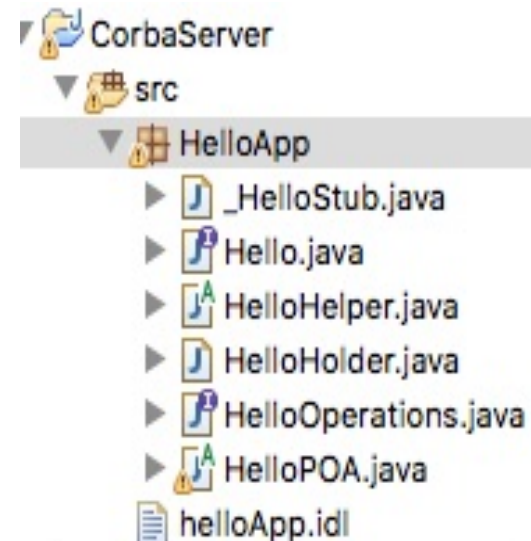
```
}; //shutdown() pour  
arrêter l'ORB
```

La compilation se fait en
ligne de commande, après
spécification du chemin:

```
Documents/workspaceJava/CorbaSe  
rver/src/> idlj -fall HelloApp.idl
```

Après la compilation, ce
module est mappé en
un package:

`package HelloApp`, qui
contient les squelettes
nécessaires pour
assembler le serveur



3.1- Implémenter le serveur(HelloServer.java)

- Le serveur consiste en 2 classes: servant et server
- Le servant: `HelloImpl` est l'implémentation de l'interface IDL `Hello`, c'est une sous classe de `HelloPOA`, qui est une des classes générées par le compilateur `idlj`.
- Le servant contient une méthode pour chaque opération IDL, dans notre exemple, `sayHello()` et `shutdown()`

3.2- Implémenter le serveur(HelloServer.java)

- La 2ème classe server (qui est en réalité HelloServer), contient la méthode main, qui aura comme tâches de:
- Créer et initialiser une instance ORB
- Obtenir une référence vers le root POA et activer le POAManager
- Créer une instance du servant et informer l'ORB la concernant.
- Obtenir une référence objet CORBA pour un naming context dans lequel elle va enregistrer le nouvel objet CORBA
- Obtenir la racine Naming context
- Enregistrer le nouvel objet dans le Naming context sous le nom « Hello »
- Attendre les invocations du nouvel objet de la part du client

Code source Servant

```
import HelloApp.*;
import org.omg.CosNaming.*;
import org.omg.CosNaming.NamingContextPackage.*;
import org.omg.CORBA.*;
import org.omg.PortableServer.*;
import org.omg.PortableServer.POA;
import java.util.Properties;
class HelloImpl extends HelloPOA {
//Créer et initialiser une instance ORB
private ORB orb;
public void setORB(ORB orb_val) { orb = orb_val; }

// implement sayHello() method
public String sayHello() { return "\nHello world !!\n"; }
// implement shutdown() method
public void shutdown() { orb.shutdown(false); } }
```

Code source Server

```
public class HelloServer {  
    public static void main(String args[]) {  
        try{  
            // create and initialize the ORB  
            ORB orb = ORB.init(args, null);  
  
            // get reference to rootpoa & activate the  
            //POAManager  
            POA rootpoa =  
            POAHelper.narrow(orb.resolve_initial_references("RootPOA"));  
            rootpoa.the_POAManager().activate();  
  
            // create servant and register it with the ORB  
            HelloImpl helloImpl = new HelloImpl();  
            helloImpl.setORB(orb);  
  
            // get object reference from the servant  
            org.omg.CORBA.Object ref =  
                rootpoa.servant_to_reference(helloImpl);  
            Hello href = HelloHelper.narrow(ref);  
  
            // get the root naming context  
            // NameService invokes the name service
```

```
            org.omg.CORBA.Object objRef =  
                orb.resolve_initial_references("NameService");  
  
            // Use NamingContextExt which is part of the  
            //Interoperable Naming Service (INS) specification.  
  
            NamingContextExt ncRef =  
                NamingContextExtHelper.narrow(objRef);  
  
            // bind the Object Reference in Naming  
  
            String name = "Hello";  
            NameComponent path[] =  
                ncRef.to_name( name );  
            ncRef.rebind(path, href);  
            System.out.println("HelloServer ready and waiting ...");  
            // wait for invocations from clients  
            orb.run(); } catch (Exception e) {  
                System.err.println("ERROR: " + e);  
                e.printStackTrace(System.out); }  
            System.out.println("HelloServer Exiting ..."); } }
```

Implémenter la classe Client

HelloClient

Cette classe doit:

- Créer et initialiser un ORB
- Obtenir une référence vers le root naming context
- Chercher « Hello » dans le naming context et recevoir une référence vers cet objet CORBA
- Invoquer les opérations sayHello() et shutdown() sur l'objet et afficher les résultats.

Code source du Client

- *HelloClient.java*

```
import HelloApp.*;
import org.omg.CosNaming.*;
import
org.omg.CosNaming.NamingContextPackage.*;
import org.omg.CORBA.*;

public class HelloClient {
    static Hello helloImpl;

    public static void main(String args[]) {
        try{
            // create and initialize the ORB
            ORB orb = ORB.init(args, null);

            // get the root naming context
            org.omg.CORBA.Object objRef =
            orb.resolve_initial_references("NameService");
            // Use NamingContextExt instead of
            NamingContextExt ncRef =
                NamingContextExtHelper.narrow(objRef);

            // resolve the Object Reference in Naming
            String name = "Hello";
            helloImpl =
                HelloHelper.narrow(ncRef.resolve_str(name));
            System.out.println("Obtained a handle on server
                object: " + helloImpl);
            System.out.println(helloImpl.sayHello());

            helloImpl.shutdown(); }
            catch (Exception e) { System.out.println("ERROR
                : " + e) ; e.printStackTrace(System.out); } } }
```

5-Etapes d'Exécution:

1-Démarrer le Naming Service

Pour démarrer orbd depuis une invite de commande Solaris, Linux, or Mac OS X :

- *orbd* -ORBInitialPort 1050 (ou *tnameserv* -ORBInitialPort 1050)

Depuis un prompt MS-DOS (Windows) :

- start orbd -ORBInitialPort 1050

NB.1050 est le port sur lequel on veut exécuter le serveur de nom . -ORBInitialPort un argument requis

Etapes d'Exécution:

2-Démarrer le Serveur

- Start the Hello server. To start the Hello server from a Solaris, Linux, or Mac OS X command shell, enter:
- `java HelloServer -ORBInitialPort 1050 -ORBInitialHost localhost`& From an MS-DOS system prompt (Windows), enter:
- `start java HelloServer -ORBInitialPort 1050 -ORBInitialHost localhost` For this example, you can omit `-ORBInitialHost localhost` since the name server is running on the same host as the Hello server. If the name server is running on a different host, use `-ORBInitialHost nameserverhost` to specify the host on which the IDL name server is running.
- Specify the name server (orbd) port as done in the previous step, for example, `-ORBInitialPort 1050`.

Etapes d'Exécution:

3-Démarrer le Client

- Run the client application: `java HelloClient -ORBInitialPort 1050 -ORBInitialHost localhost` For this example, you can omit `-ORBInitialHost localhost` since the name server is running on the same host as the Hello client. If the name server is running on a different host, use `-ORBInitialHost nameserverhost` to specify the host on which the IDL name server is running.
- Specify the name server (orbd) port as done in the previous step, for example, `-ORBInitialPort 1050`.
- The client prints the string from the server to the command line: Hello world!!